

Queues: Overview and Objectives

An abstract data type built on first in, first out order

1. What this unit is about

A queue is another abstract data type, alongside the linked list and the stack from the last two units. The same wall/information-hiding idea from the linked list unit applies here: a queue offers a set of operations, and this unit looks at two different ways to carry those operations out behind the wall, an array-based version and a linked version, built on the node and link ideas you already know.

A queue holds its items in first in, first out order, often shortened to FIFO. Items are added at one end, called the rear or back, and removed from the other end, called the front. The two core operations are enqueue, which adds an item at the rear, and dequeue, which removes and returns the item at the front. A plain array-based queue runs into a practical problem as items are removed from the front and added at the rear, so this unit covers the circular queue, which reuses freed array slots by wrapping the front and rear indices back to the start of the array. Queues show up constantly once you know to look for them: a print queue, a line of customers, task scheduling, and, later in the course, the order in which a graph is explored level by level.

As with the earlier units, we work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. The queue idea is identical in both, and both the array-based (circular) and linked implementations appear in each language.

Class time favors drawings and live code over slides. Read ahead, try tracing an enqueue and dequeue sequence on paper yourself, and come ready to think out loud.

2. Learning objectives

By the end of the unit you should be able to:

Remember and understand

- State that a queue is an ADT, and recall the wall/information-hiding idea from the linked list unit as it applies to a queue.
- Define a queue and state its defining property, first in, first out order.
- Discuss the standard queue operations: enqueue, dequeue, front (or peek), isEmpty, and size.
- Discuss the two implementations covered in this unit, a circular array-based queue and a linked queue, and recall the node and link vocabulary from the linked list unit as it applies to a linked queue.
- Explain why a plain, non-circular array-based queue wastes space over time, and state how a circular queue solves that problem.

- List common uses of a queue, including print queues, task scheduling, and level-by-level traversal of a graph.

Analyze

- Compare the circular array-based and linked implementations of a queue, including how each handles growth, memory use, and the cost of enqueue and dequeue.
- Explain how the front and rear indices move around a circular array, including how the queue tells full from empty when the indices meet.
- Trace a queue by hand through a sequence of enqueue and dequeue operations, and state its contents, front, and rear at each step.
- Judge whether a queue is the right ADT for a given problem, as opposed to a stack or another structure already covered.

Apply

- Diagram a queue by hand, showing the front, the rear, and the effect of an enqueue or a dequeue.
- Code queue operations from scratch in both a circular array-based version and a linked version, in your language.
- Apply a queue to a standard problem, such as simulating a line with arrivals and departures, or processing tasks in the order they were submitted.
- Generate a queue-based code solution to a new problem.

Evaluate

- Select the more appropriate queue implementation, circular array-based or linked, for a given scenario, and justify the choice against the alternative.